



MAC-106 Project Presentation

McDonalds

(Fast food franchise)

Database Management System

TEAM MEMBERS

1

Anshika Gupta

2

Kasak

3

Lavanya Nigam

4

Toshya

5

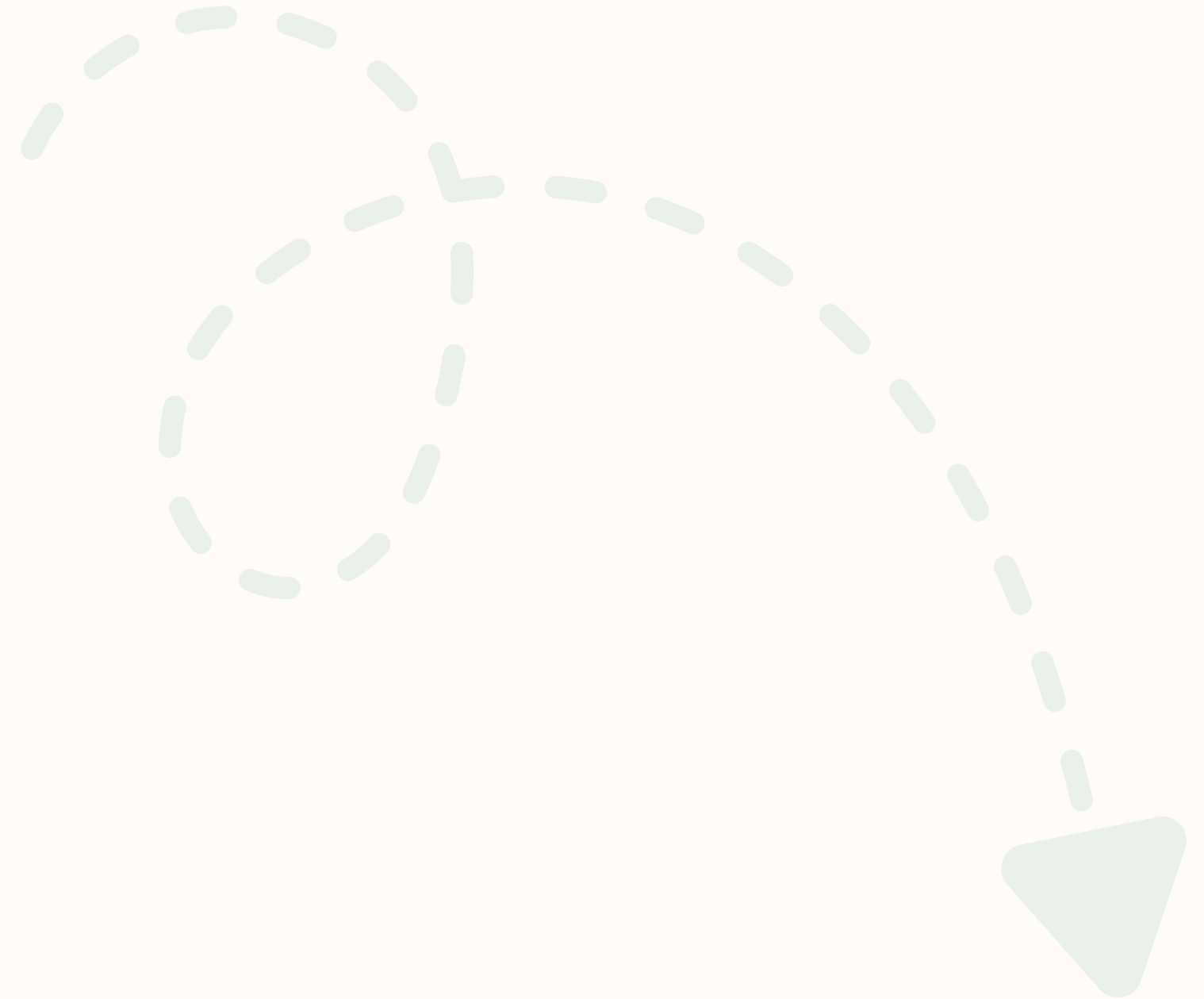
Gaurangi Vishwakarma

6

Yachana

Requirements

This database system manages the customer and staff records of a food chain franchise, designed to track the customer information and their order, staff and branch information and inventory details



ER Diagram

The Entity-Relationship (ER) Model is a visual tool used in database design to represent data and its relationships in a clear and organized manner.

It helps in identifying the key components of a system:

Entities: These are objects or concepts about which data is stored, such as Customer, staff, or order.

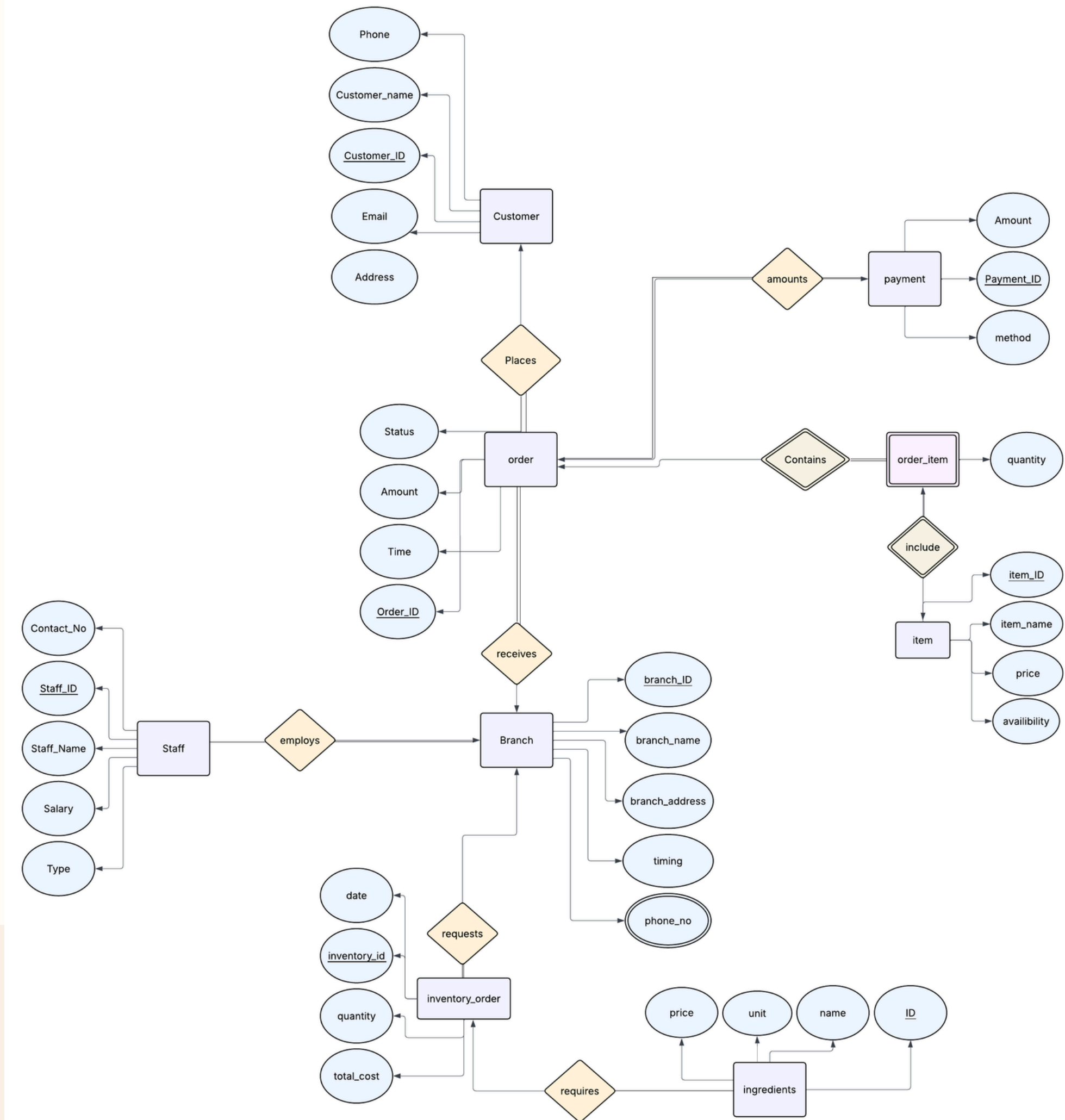
Attributes: These are details or properties of entities, like a Customer_ID and Email.

Relationships: These illustrate how entities are connected, for example, a Customer places order

By using ER diagrams, designers can create a blueprint of the database structure, ensuring that all necessary data and their interconnections are accurately captured before actual implementation.

ENTITIES

1. Staff: Relevant information about the workers of the store
2. Customer: List of all the customers and their information
3. order: orders given by the customers
4. Branch: All the working branches of the franchise
5. Inventory_order: All the relevant information about the resources used by the branch
6. Payment: Information about the payment done by the customers
7. item: Tells about the items in the menu
8. ingredients: Information about all the ingredients used by the branch
9. order_item: A weak entity set with identifying set as item. It tells about the items contained in the order



ENTITY SET

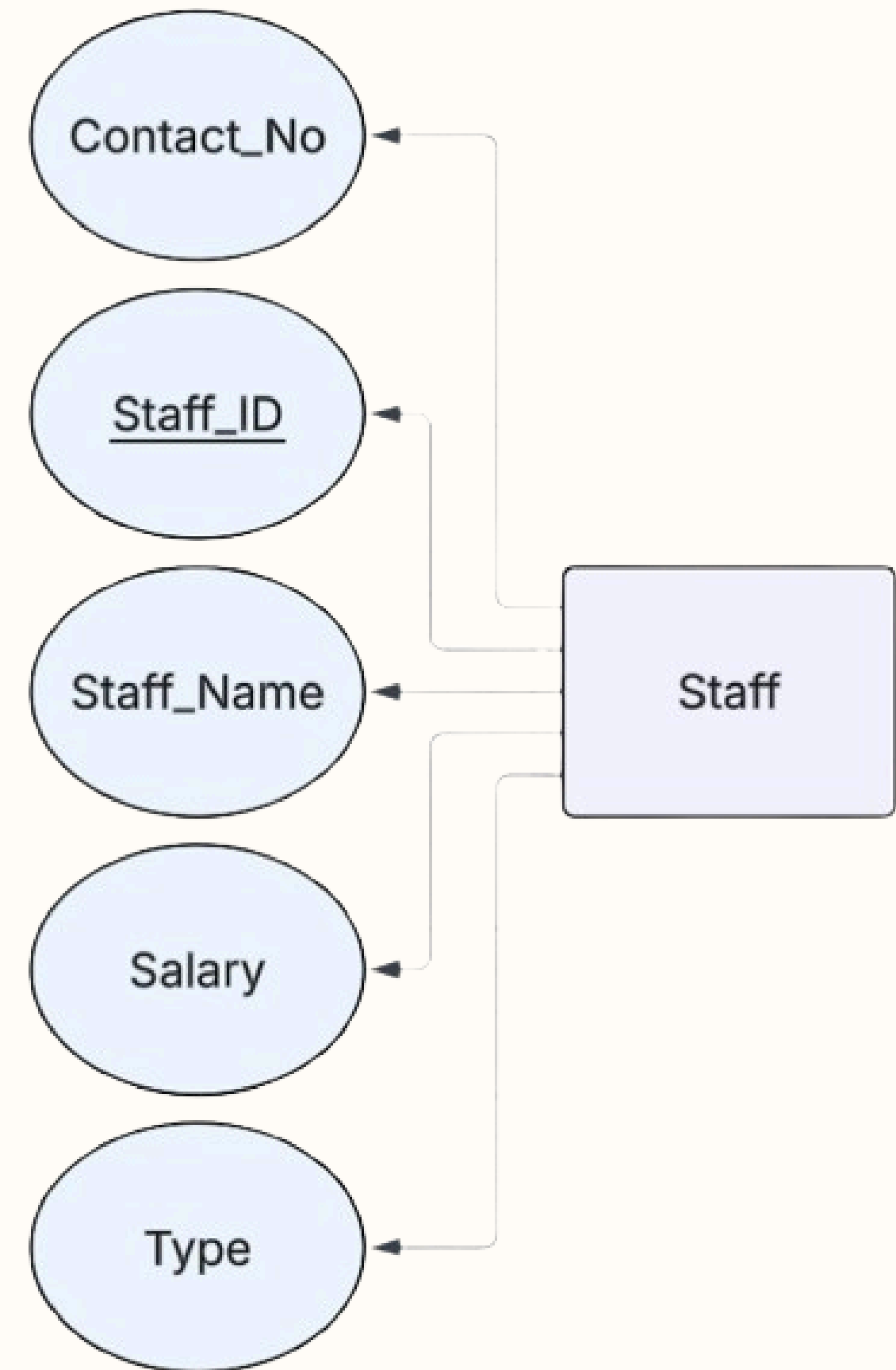
Staff entity

1. Staff_ID (Primary Key)
2. Staff_Name
3. Contact_No
4. Salary
5. Type
6. branch_ID (Foreign Key)

KEYS

Staff_ID (PK)

branch_ID (FK)



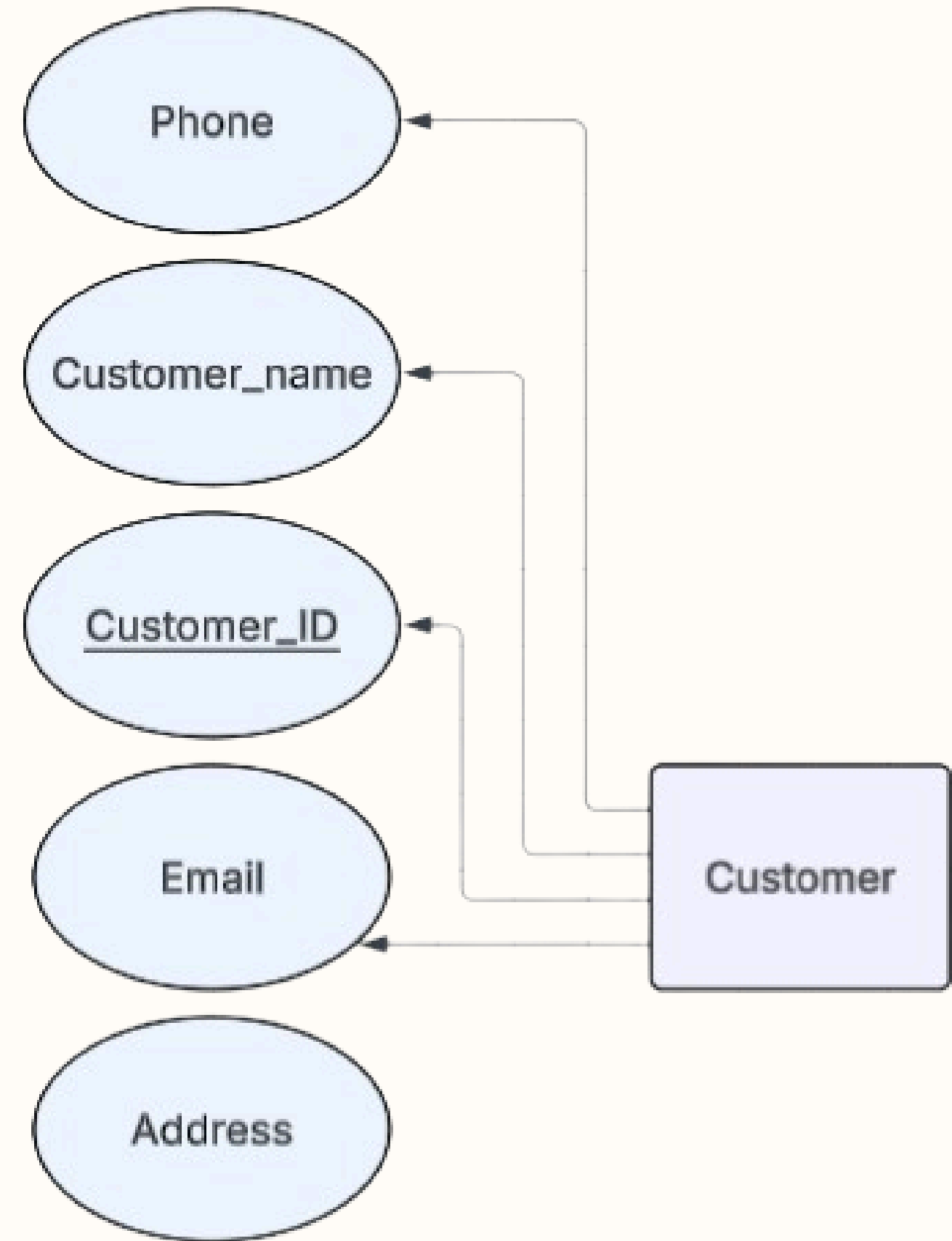
ENTITY SET

Customer entity

1. Customer_ID (Primary Key)
2. Customer_name
3. Phone
4. Email
5. Address

KEYS

Customer_ID (PK)



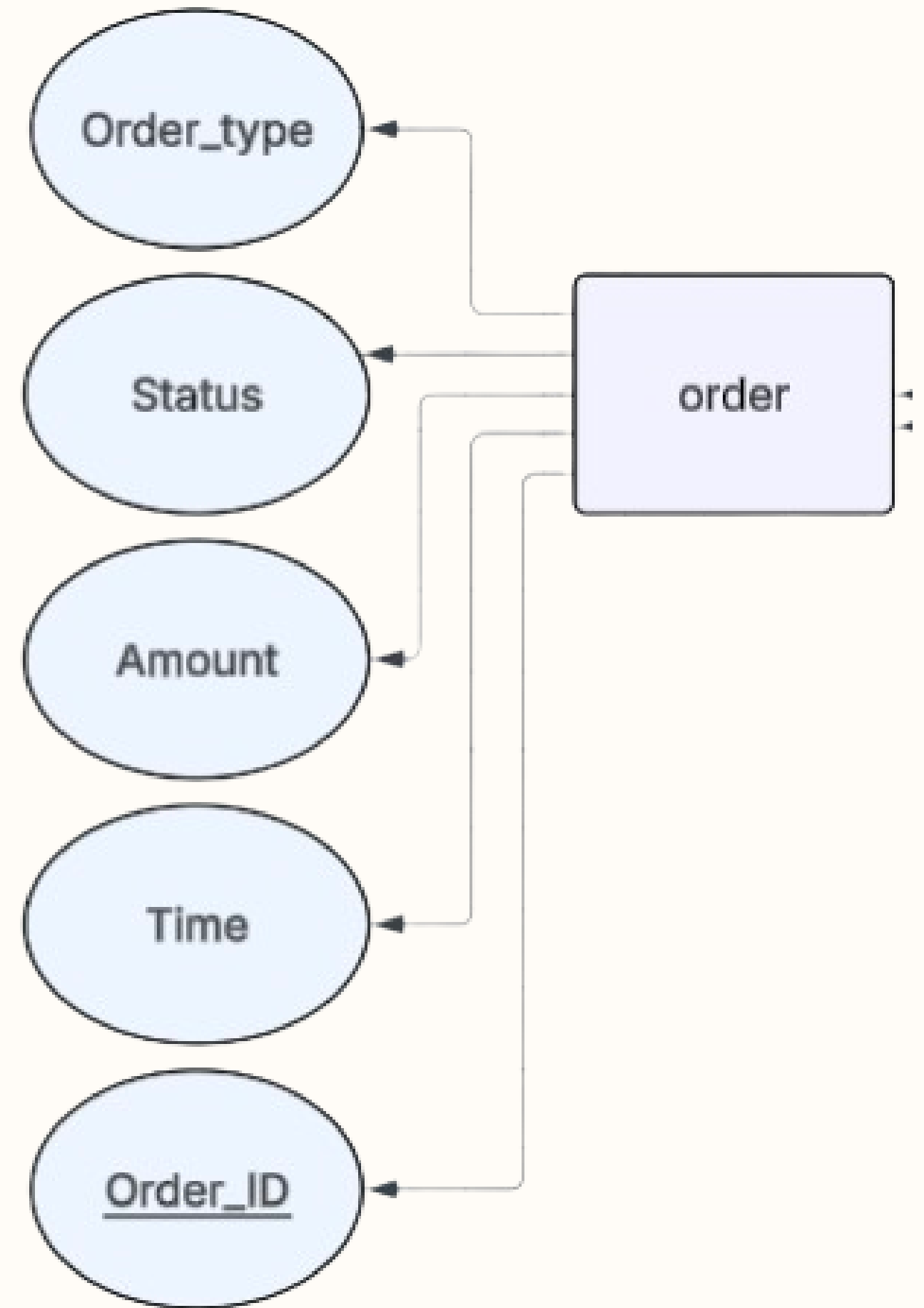
ENTITY SET

Order entity

1. Order_ID (Primary Key)
2. Time
3. Status
4. Amount
5. Customer_ID (Foreign Key)
6. branch_ID (Foreign Key)

KEYS

Order_ID (PK)
Customer_ID (FK)
branch_ID (FK)



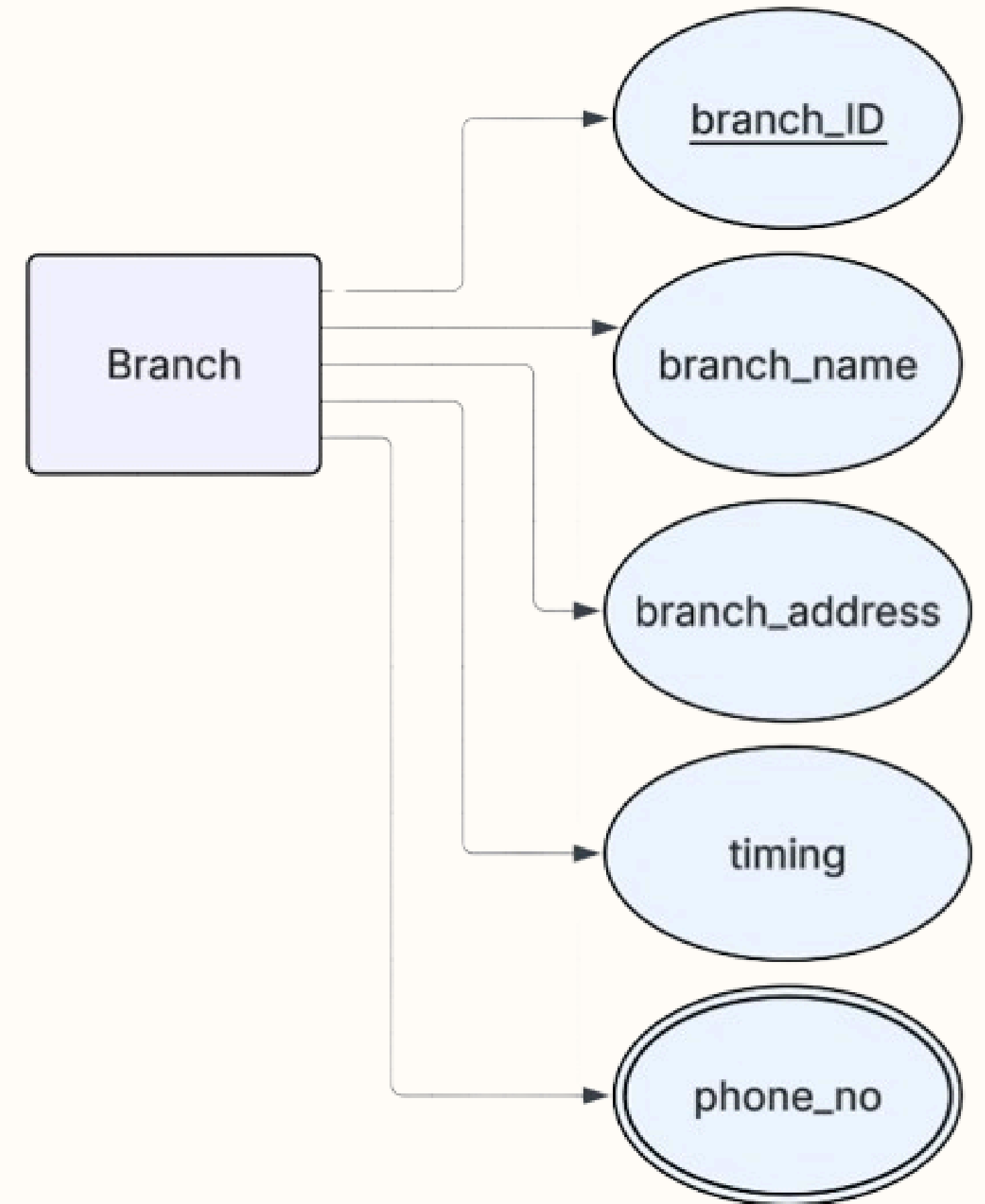
ENTITY SET

Branch entity

1. branch_ID (Primary Key)
2. branch_name
3. branch_address
4. timing
5. phone_no

KEYS

branch_ID (PK)



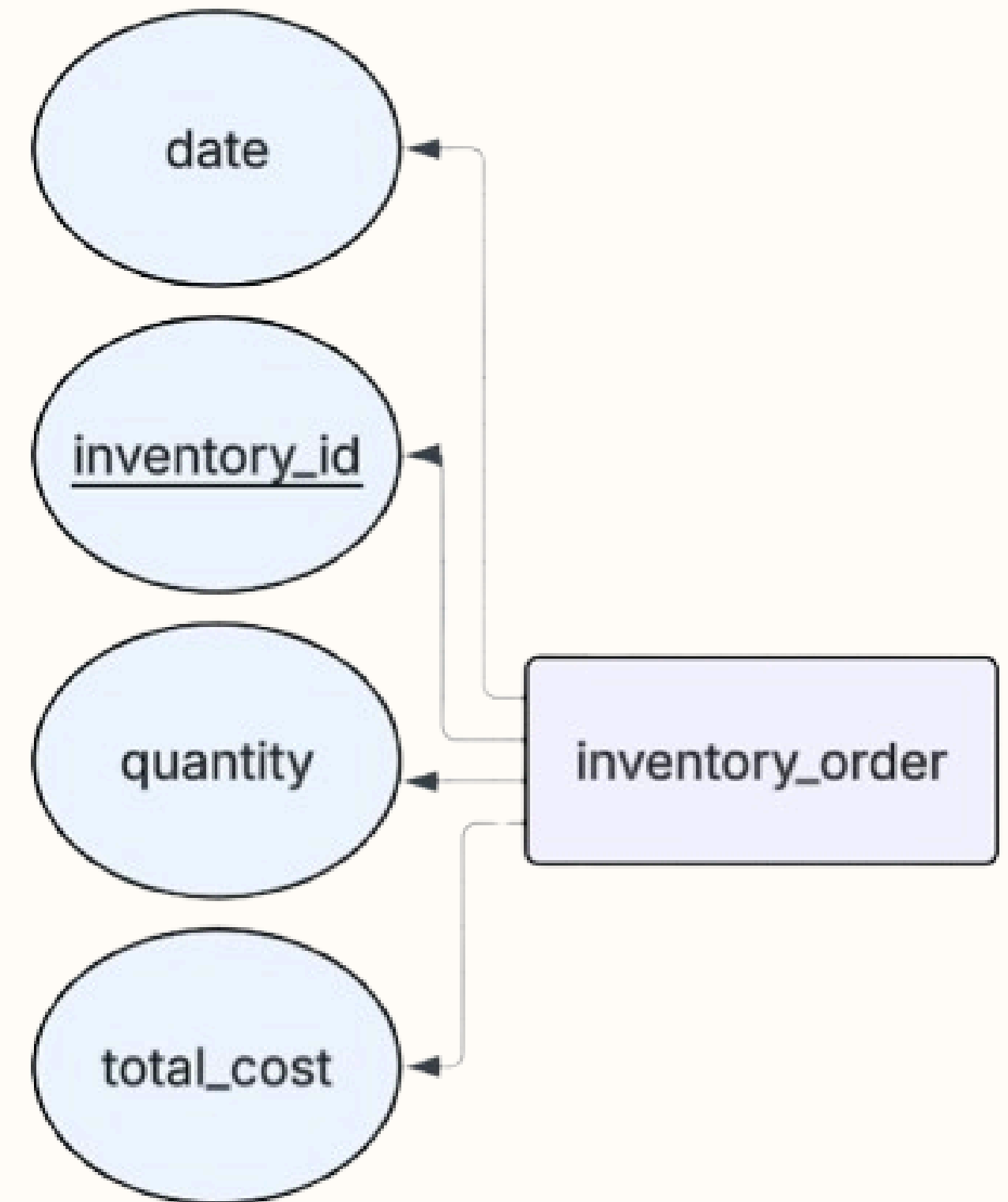
ENTITY SET

Inventory_order entity

- 1.inventory_id (Primary Key)
- 2.date
- 3.quantity
- 4.total_cost
- 5.branch_ID (Foreign Key)

KEYS

inventory_id (PK)
branch_ID (FK)



ENTITY SET

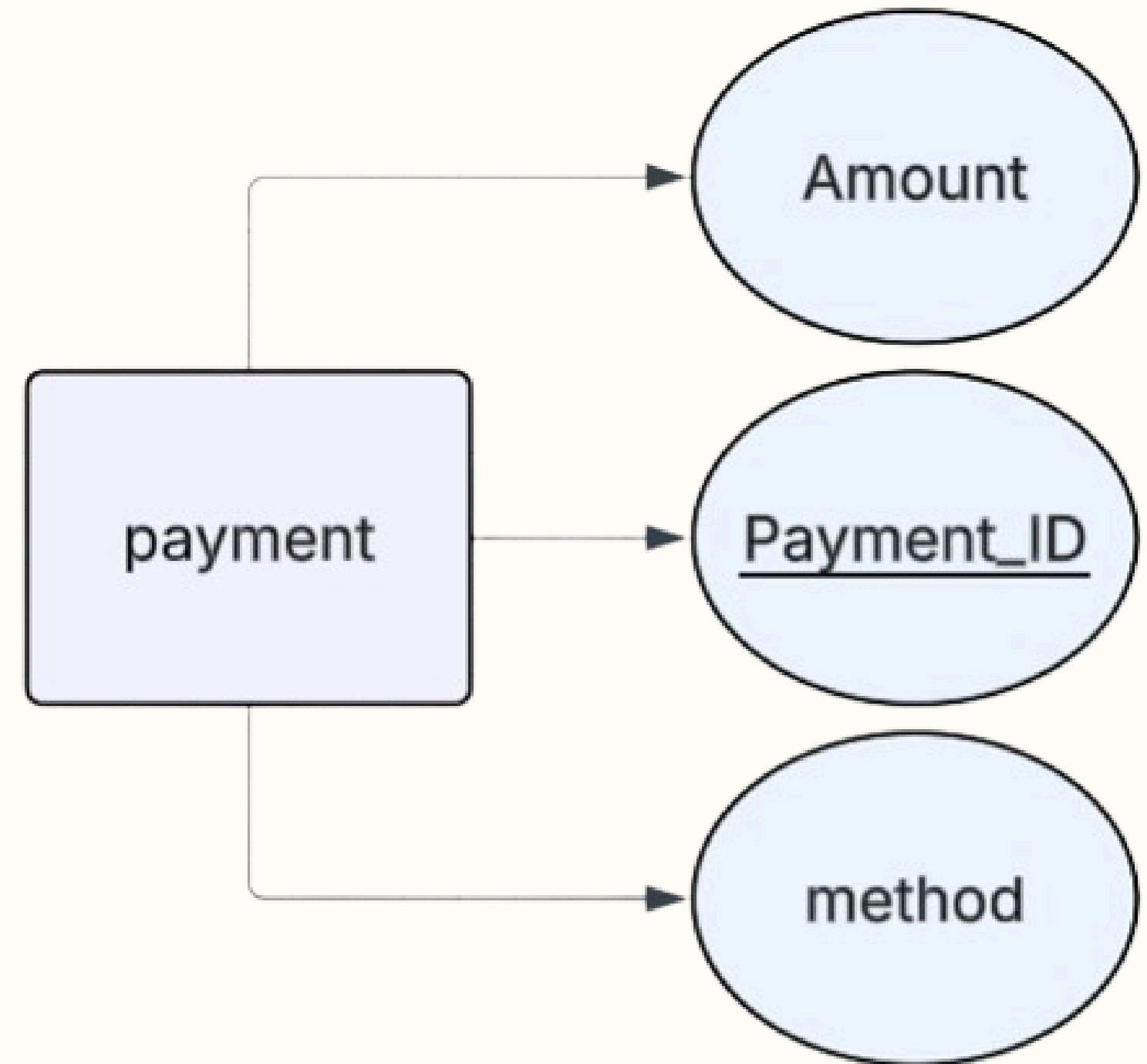
payment entity

- 1.Payment_ID (Primary Key)
- 2.Amount
- 3.method
- 4.Order_ID (Foreign Key)

KEYS

Payment_ID (PK)

Order_ID (FK)



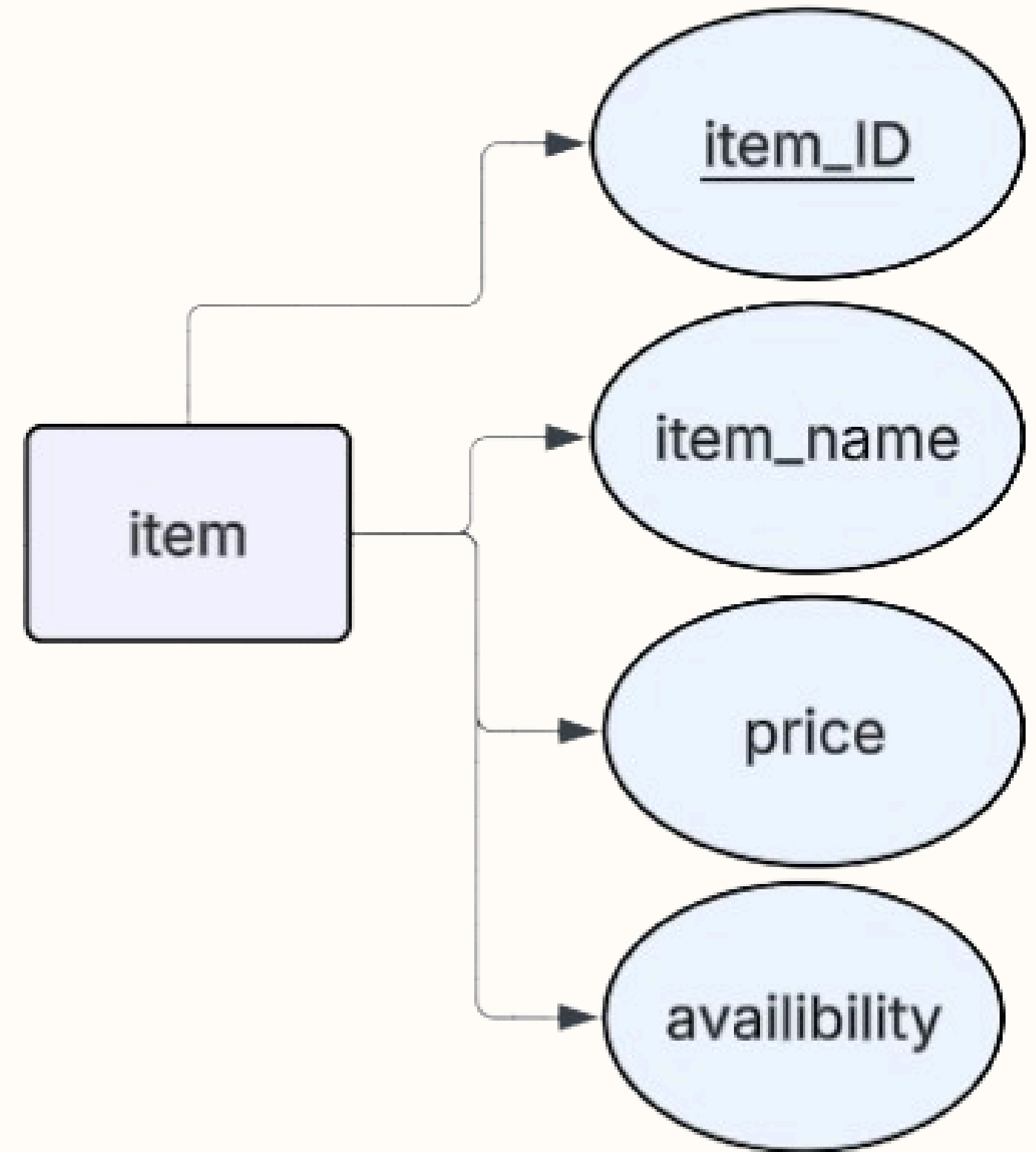
ENTITY SET

item entity

- 1.item_ID (Primary Key)
- 2.item_name
- 3.price
- 4.availability

KEYS

item_ID (PK)



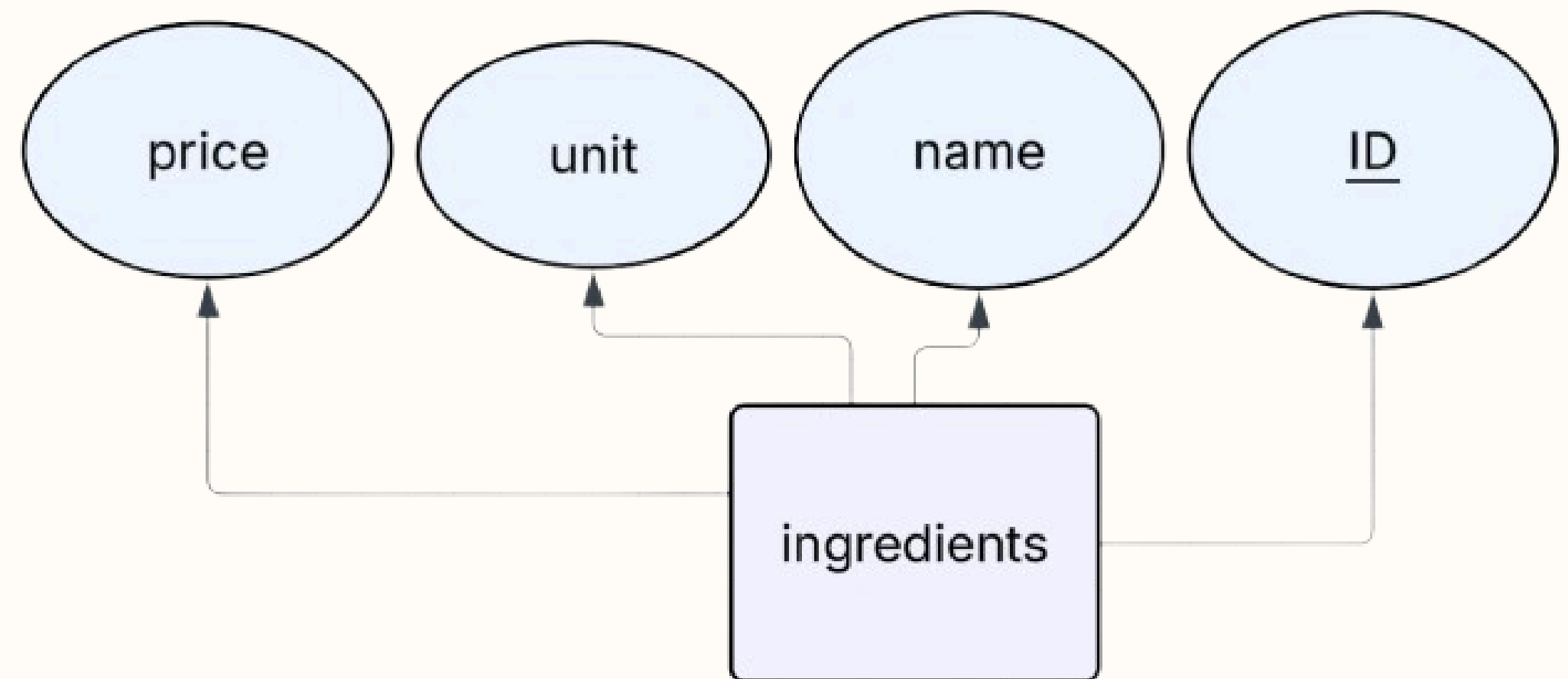
ENTITY SET

ingredients entity

- 1.ID (Primary Key)
- 2.name
- 3.unit
- 4.price

KEYS

ID (PK)



Weak Entity Set

In the Entity-Relationship (ER) model, a weak entity set is an entity that cannot be uniquely identified by its own attributes alone.

It depends on a related strong entity—an entity with a primary key—for its identification.

To uniquely identify a weak entity, a partial key (also known as a discriminator) is combined with the primary key of the strong entity.

In the final Table for Relational Model, the Weak Entity and Weak Relationship are combined into a Table with Primary Key and Partial Key as the Composite Key

ENTITY SET

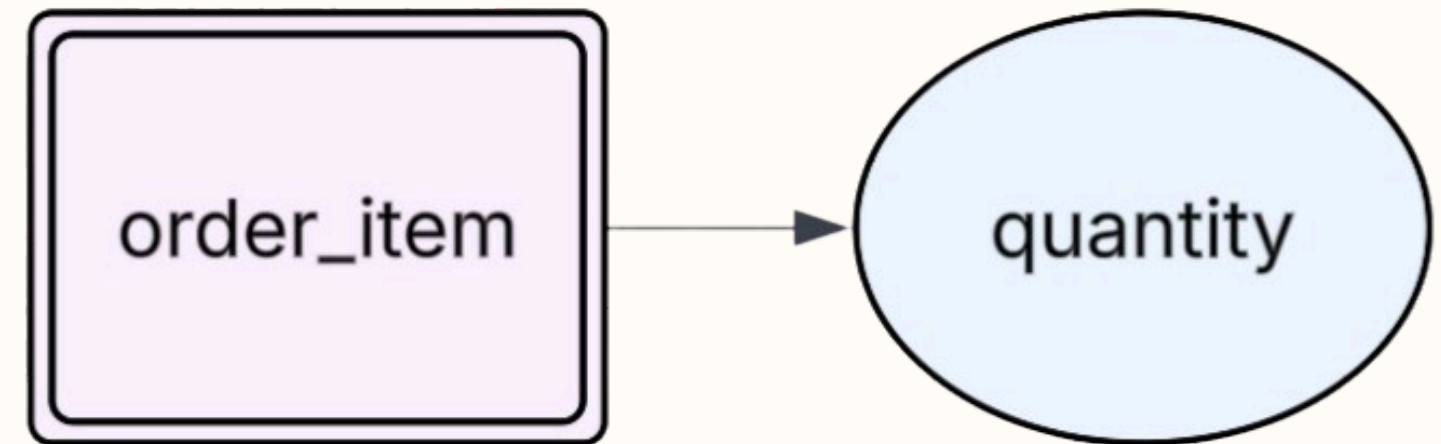
order_item entity (Weak Entity Set)

1. Order_ID (Foreign Key)
2. item_ID (Foreign Key)
3. quantity

The identifying entity set is item

KEYS

item_ID (FK)



ER Diagram

Relations



A relation is an
association amongst
two or more entity sets

It is represented by a
diamond shape in the ER
Diagram

employs relationship

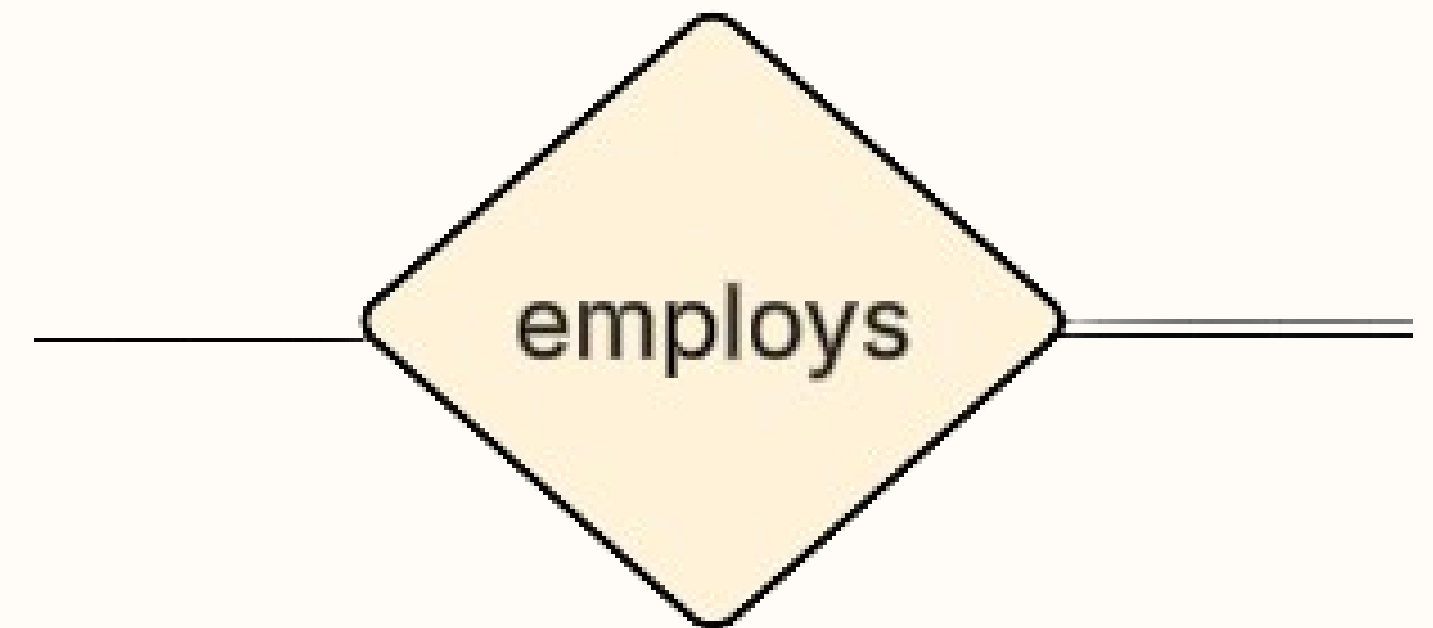
It joins Staff and Branch

It is a many-one relationship with staff as the many side and Branch as the one side.

It involves total participation from the branch side

KEYS

Staff_ID



Places relationship

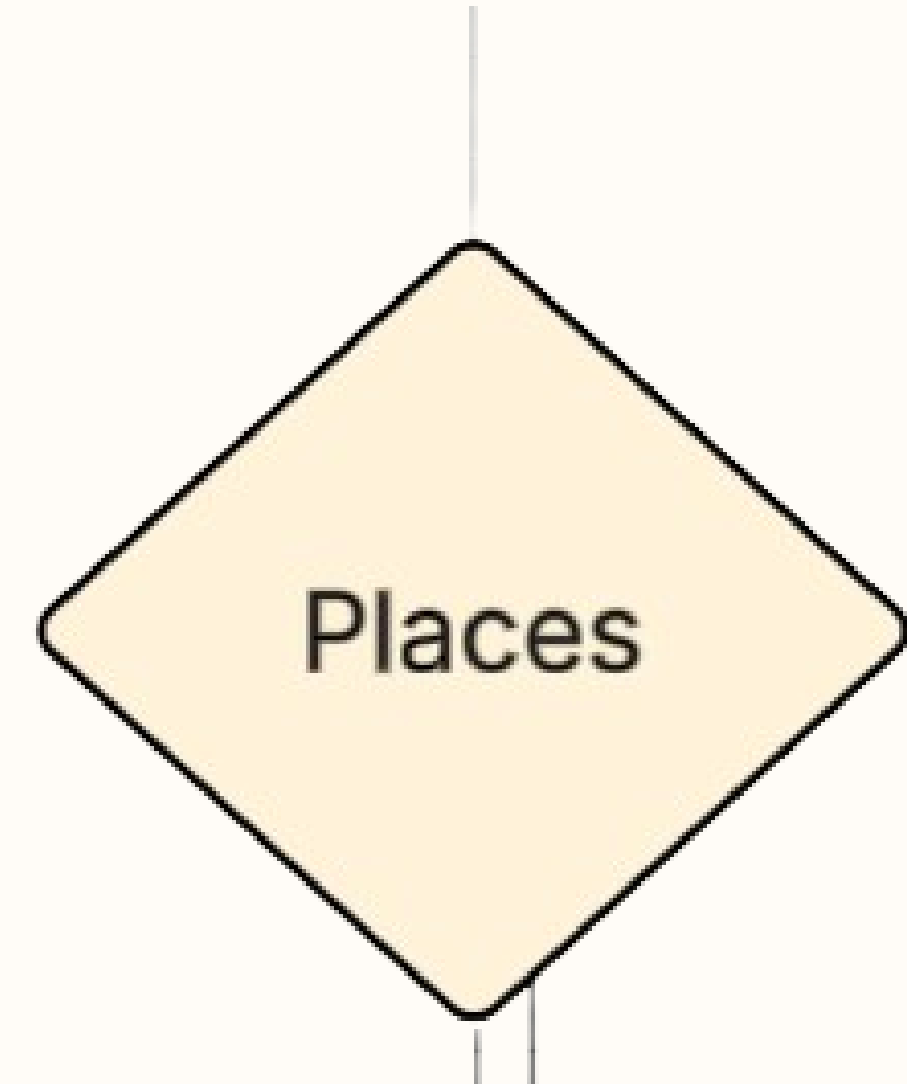
It joins Customer and order

It is a one-many relationship with order as the many side and Customer as the one side.

It involves total participation from the order side

KEYS

Order_ID



receives relationship

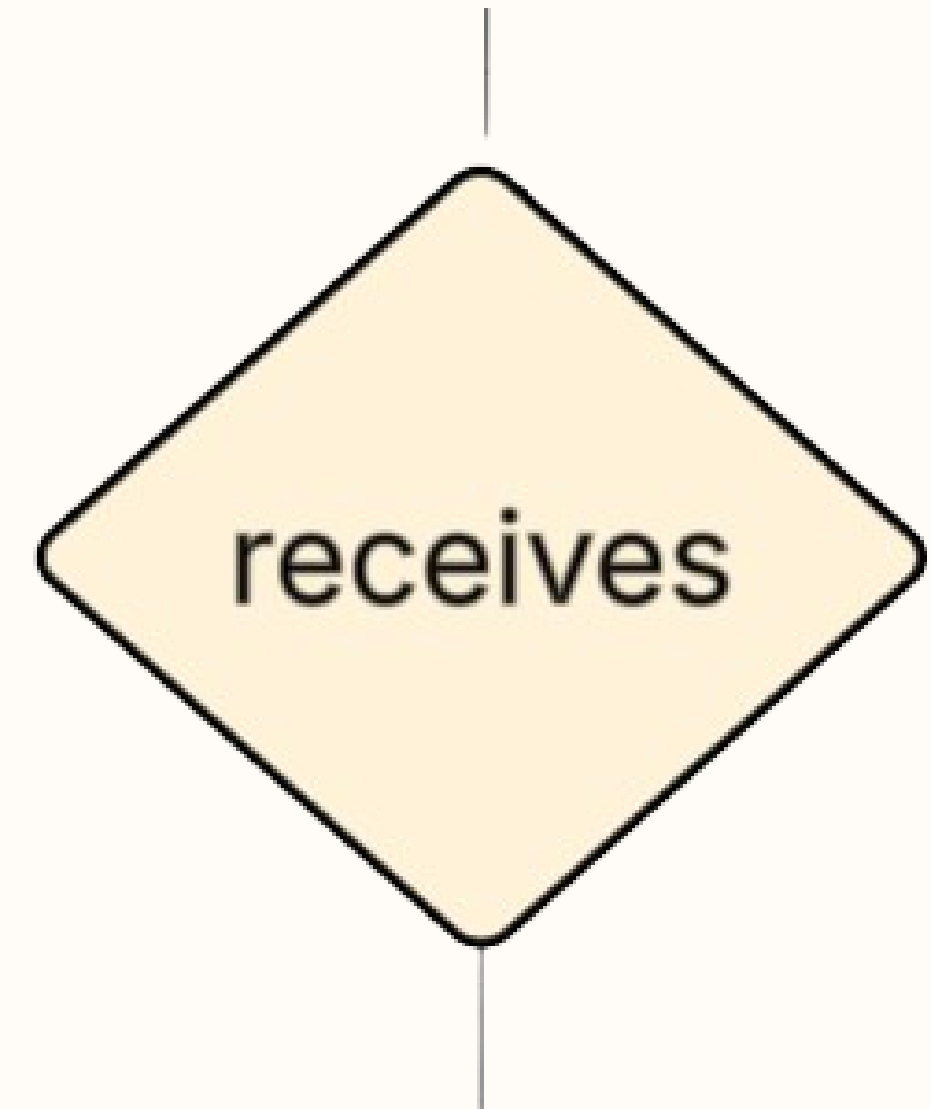
It joins Branch and order

It is a one-many relationship with order as the many side and Branch as the one side.

It involves total participation from the order side

KEYS

Order_ID



requests relationship

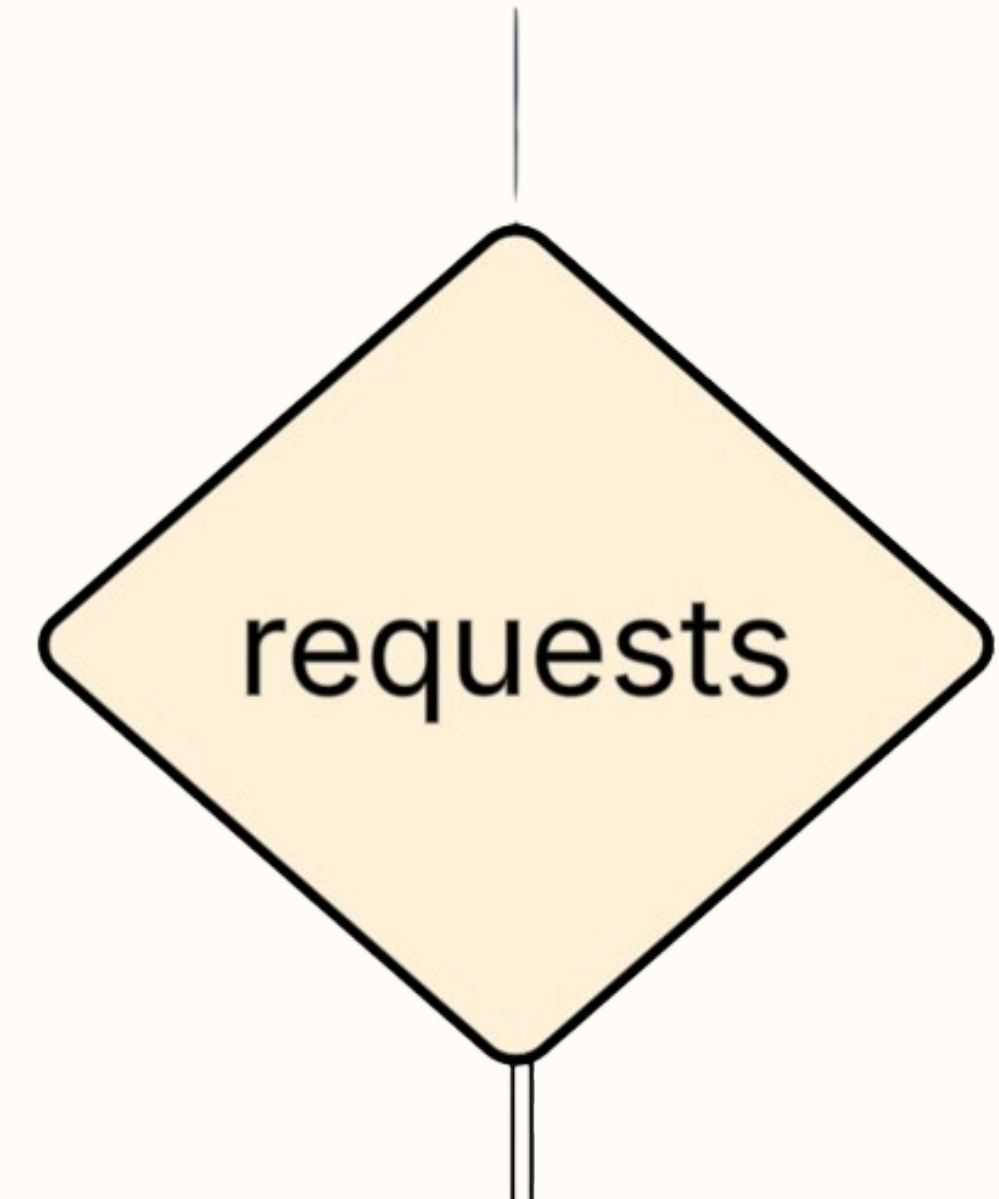
It joins Branch and inventory_order

It is a one-many relationship with inventory_order as the many side and Branch as the one side.

It involves total participation from the inventory_order side

KEYS

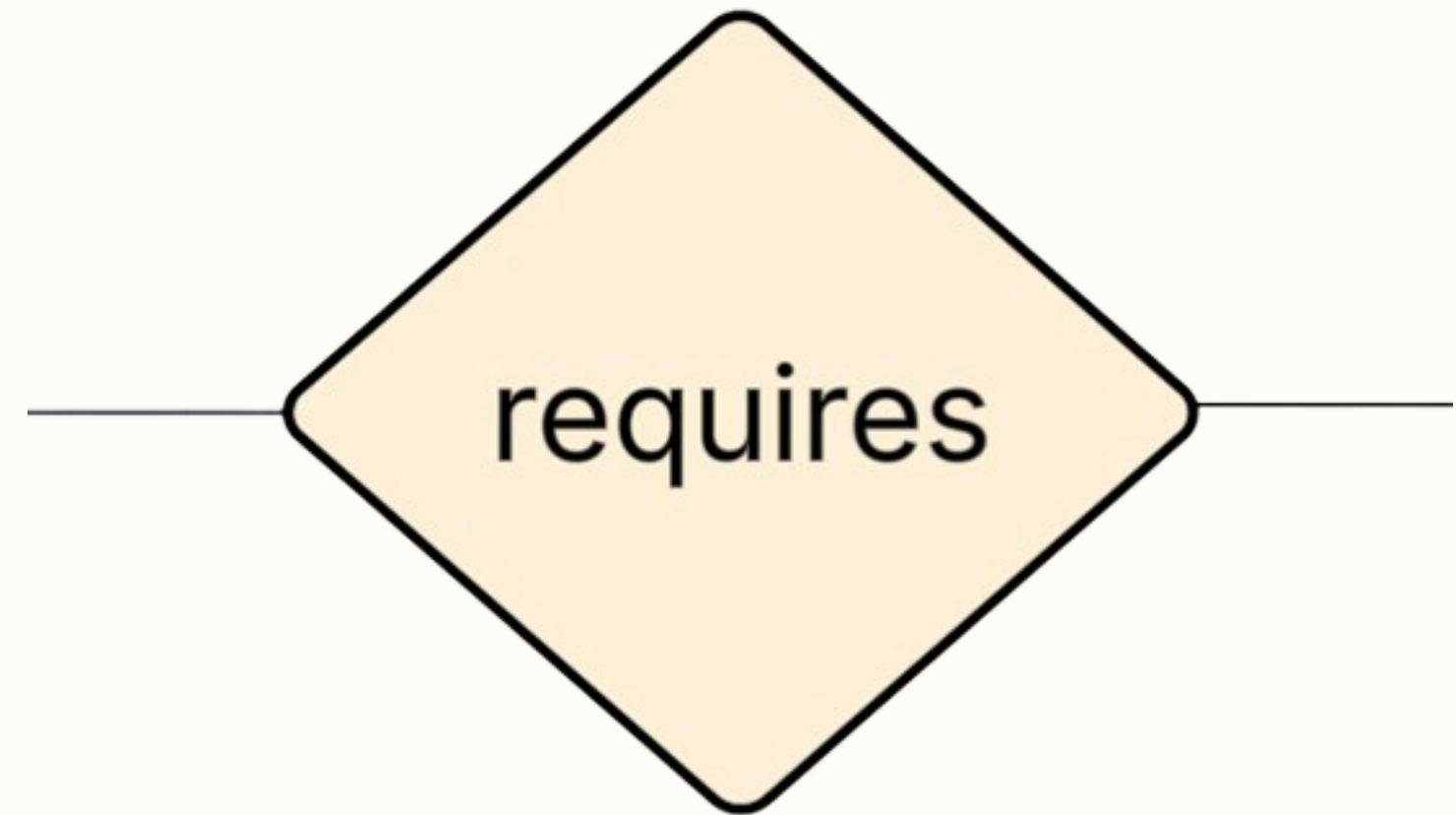
inventory_id



requires relationship

It joins ingredients and inventory_order
It is a one-many relationship with ingredients as
the many side and inventory_order as the one
side.

KEYS
ID



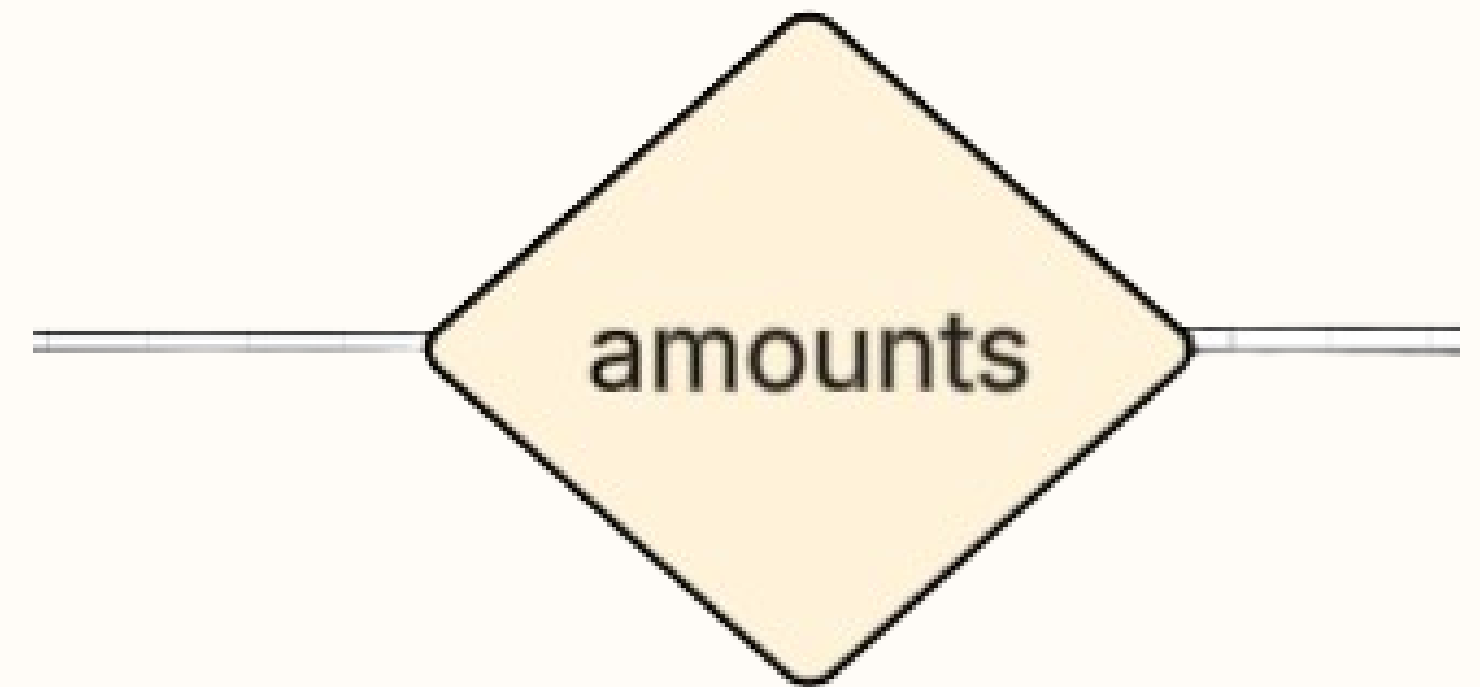
amounts relationship

It joins order and payment.

It is a one-one relationship. It involves total participation from both entity sets

KEYS

Composite Key: Order_ID, Payment_ID

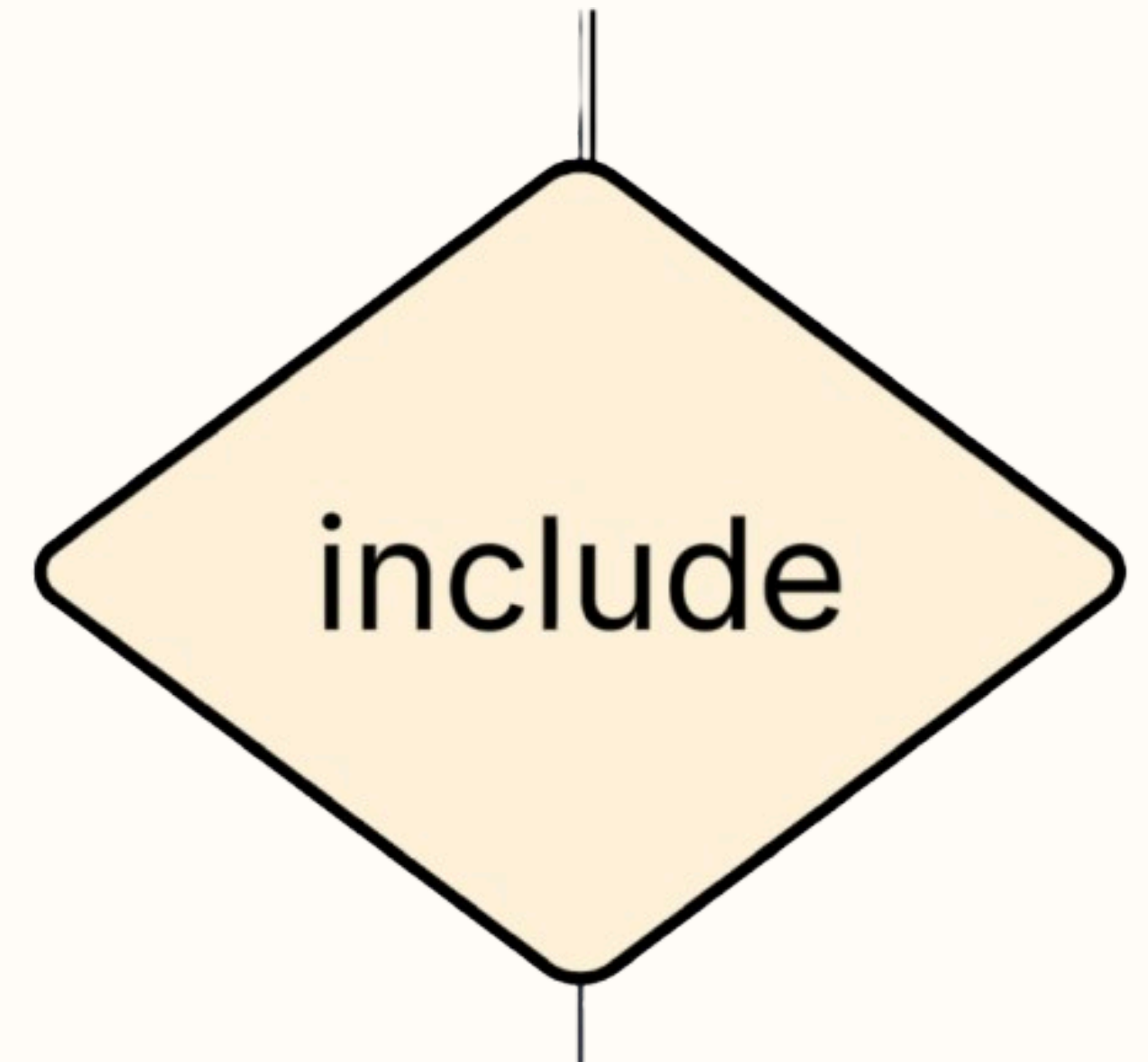


include relationship

It joins order_item and item.

It is a one-one relationship. It involves total participation from order_item entity set.

KEYS
item_ID



Contains relationship

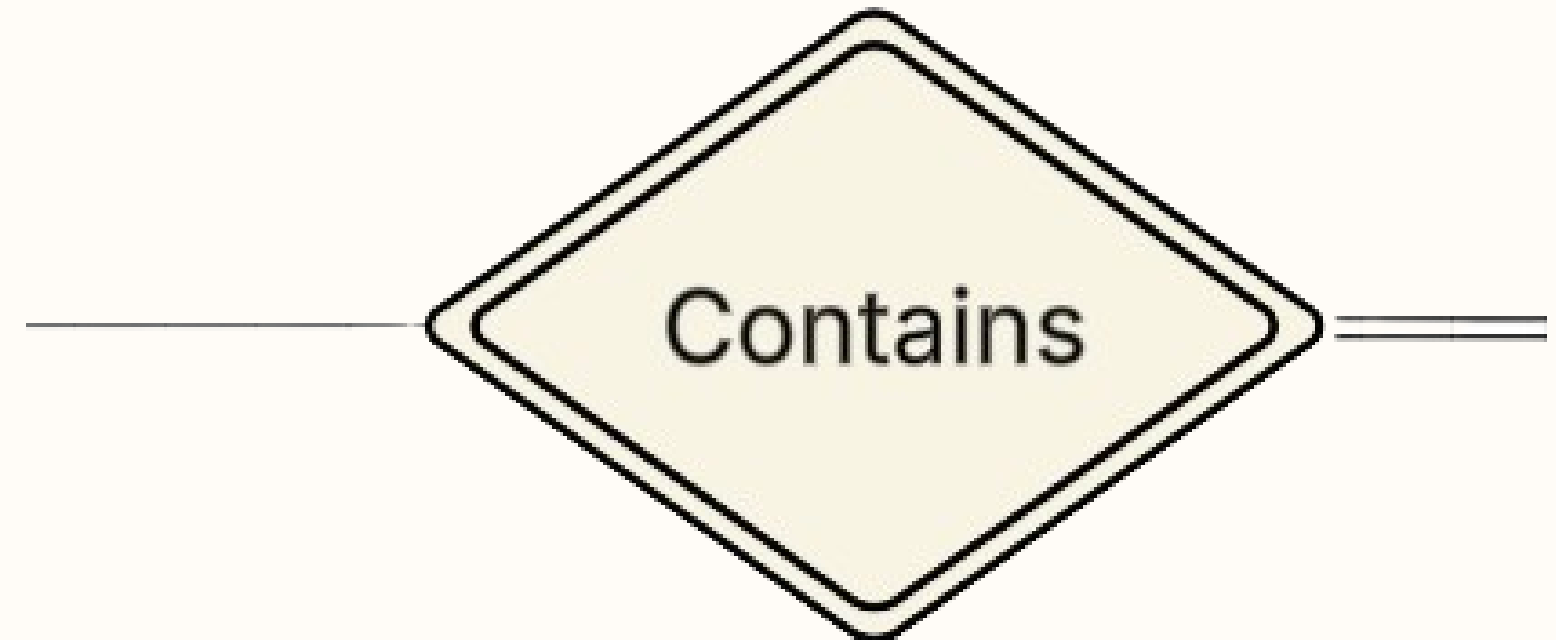
Weak Relationship

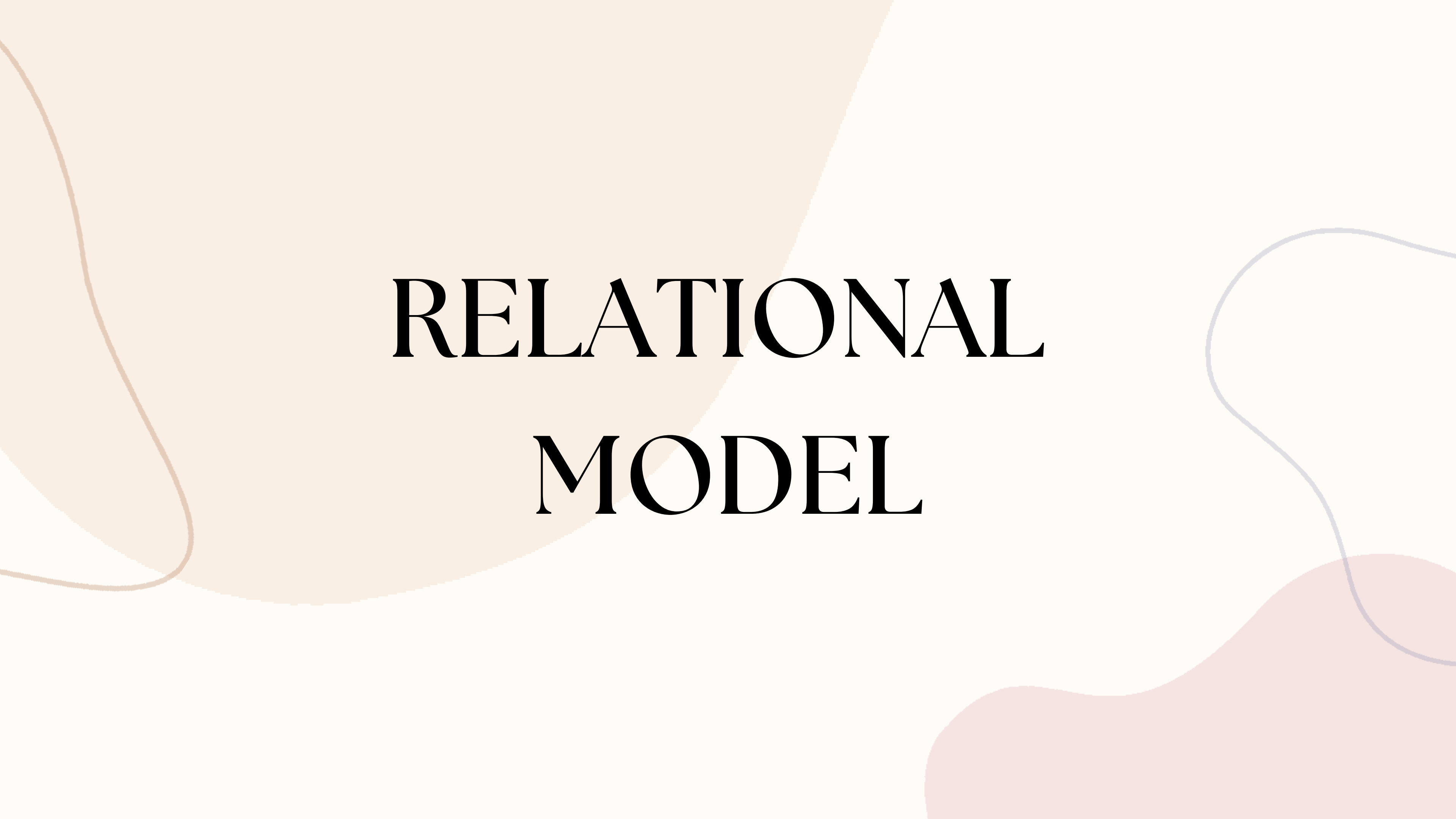
It joins order_item and order.

It is a one-many relationship with order as the one side and order_item as the many side. It involves total participation from order_item entity set.

KEYS

Order_ID





RELATIONAL MODEL

Relational model

* The relational model is a database model that organizes data in the form of tables (relations).

1. Structure of Table

- * Rows (Tuples): Represent individual records.
- * Columns (Attributes): Represent properties or fields of data.

2. Key Concepts

- * Primary Key: Uniquely identifies each record in a table.
- * Foreign Key: Connects one table to another.
- * Relation: A complete table.

3. Properties

- * No duplicate rows.
- * Each column has a unique name.
- * Data is stored in a structured format.

4. Advantages

- * Simple and easy to understand.
- * Ensures data integrity and consistency.
- * Flexible and widely used in modern databases.



Customer Table

```
CREATE TABLE Customer (  
  Customer_ID INT PRIMARY KEY,  
  Customer_Name VARCHAR(50),  
  Email VARCHAR(100) UNIQUE,  
  CHECK (Email LIKE '%@%.%'),  
  Phone VARCHAR(10) UNIQUE,  
  CHECK (LENGTH(Phone) = 10),  
  Address VARCHAR(200)  
);
```

Customer_ID	Customer_Name	Email	Phone	Address
1	Aman Sharma	aman@gmail.com	9876543210	Delhi
2	Priya Verma	priya@yahoo.com	9123456780	delhi
3	Rahul Singh	rahul@gmail.com	9988776655	Delhi
4	Sneha Gupta	sneha@gmail.com	9871234567	Delhi
5	Karan Mehta	karan@yahoo.com	9090909090	Delhi
6	Neha Joshi	neha@gmail.com	9898989898	Delhi

Branch Table

```
CREATE TABLE Branch (  
  Branch_ID INT PRIMARY KEY,  
  Branch_Name VARCHAR(50),  
  Branch_Address VARCHAR(200),  
  Phone_No VARCHAR(10),  
  Timing VARCHAR(50)  
);
```

	Branch_ID	Branch_Name	Branch_Address	Phone_No	Timing
▶	1	North Branch	Delhi	9876543211	9 AM - 10 PM
	2	South Branch	Delhi	9876543212	10 AM - 11 PM
	3	East Branch	Delhi	9876543213	9 AM - 9 PM
	4	West Branch	Delhi	9876543214	8 AM - 10 PM
	5	Central Branch	Delhi	9876543215	10 AM - 10 PM
	6	City Branch	Delhi	9876543216	9 AM - 11 PM
	7	Metro Branch	Delhi	9876543217	8 AM - 9 PM

Staff Table

```
• CREATE TABLE staff (  
    Staff_ID INT PRIMARY KEY,  
    staff_name VARCHAR(100) NOT NULL,  
    contact_no VARCHAR(15) UNIQUE,  
    Type VARCHAR(50),  
    salary DECIMAL(10,2),  
    branch_ID INT,  
    FOREIGN KEY (branch_ID) REFERENCES branch(branch_ID)  
);
```

	Staff_ID	Staff_Name	Contact_No	Type	Salary	Branch_ID
▶	1	Rohan Sharma	9876543201	Manager	35000.00	1
	2	Priya Verma	9876543202	Chef	28000.00	2
	3	Aman Singh	9876543203	Waiter	18000.00	3
	4	Sneha Gupta	9876543204	Cashier	22000.00	4
	5	Karan Mehta	9876543205	Chef	30000.00	5

Payment Table

```
CREATE TABLE Payment (  
  Payment_ID INT PRIMARY KEY,  
  Order_ID INT,  
  Amount DECIMAL(10,2),  
  Method VARCHAR(30),  
  
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
);
```

	Payment_ID	Order_ID	Amount	Method
▶	1	1	450.00	UPI
	2	2	320.00	Cash
	3	3	560.00	Card
	4	4	280.00	UPI
	5	5	390.00	Cash
	6	6	610.00	Card
	7	7	470.00	UPI

Order Table

```
CREATE TABLE Orders (  
  Order_ID INT PRIMARY KEY,  
  Customer_ID INT,  
  Branch_ID INT,  
  Order_Type VARCHAR(30),  
  Status VARCHAR(30),  
  Amount DECIMAL(10,2),  
  Time TIMESTAMP,  
  
  FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
  FOREIGN KEY (Branch_ID) REFERENCES Branch(Branch_ID)  
);
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:							
	Order_ID	Customer_ID	Branch_ID	Order_Type	Status	Amount	Time
▶	1	1	1	Dine-In	Completed	450.00	2026-04-20 12:30:00
	2	2	2	Takeaway	Pending	320.00	2026-04-20 13:15:00
	3	3	3	Online	Completed	560.00	2026-04-20 14:00:00
	4	4	4	Dine-In	Cancelled	280.00	2026-04-20 15:10:00
	5	5	5	Takeaway	Completed	390.00	2026-04-20 16:00:00
	6	6	6	Online	Pending	610.00	2026-04-20 17:20:00
	7	7	7	Dine-In	Completed	470.00	2026-04-20 18:45:00

Item Table

```
CREATE TABLE Item (  
    Item_ID INT PRIMARY KEY,  
    Item_Name VARCHAR(50),  
    Price DECIMAL(10,2),  
    Availability VARCHAR(30)  
);
```

	Item_ID	Item_Name	Price	Availability
▶	1	Burger	120.00	Available
	2	Pizza	250.00	Available
	3	Peri Peri Fries	80.00	Available
	4	Sandwich	90.00	Available
	5	Cold Coffee	150.00	Available
	6	French Fries	110.00	Available

Order_items Table

```
CREATE TABLE Order_Items (  
  Order_ID INT,  
  Item_ID INT,  
  Quantity INT,  
  
  PRIMARY KEY (Order_ID, Item_ID),  
  
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),  
  FOREIGN KEY (Item_ID) REFERENCES Item(Item_ID)  
);
```

	Order_ID	Item_ID	Quantity
►	1	1	2
	2	3	1
	3	2	2
	4	4	1
	5	5	3
	6	6	2

Ingredients Table

```
CREATE TABLE Ingredients (
```

```
  ID INT PRIMARY KEY,
```

```
  Name VARCHAR(50),
```

```
  Unit VARCHAR(20),
```

```
  Price DECIMAL(10,2)
```

```
);
```

	ID	Name	Unit	Price
▶	1	Tomato	Kg	40.00
	2	Cheese	Kg	350.00
	3	Bread	Packet	50.00
	4	Milk	Litre	60.00
	5	Potato	Kg	30.00
	6	Coffee Powder	Packet	120.00

Inventory_Orders Table

```
CREATE TABLE Inventory_Order (  
  inventory_ID INT PRIMARY KEY,  
  date DATE,  
  quantity INT,  
  total_cost DECIMAL(10,2),  
  branch_ID INT,  
  FOREIGN KEY (branch_ID) REFERENCES Branch(branch_ID)  
);
```

	inventory_ID	date	quantity	total_cost	branch_ID
▶	1	2026-04-20	50	5000.00	1
	2	2026-04-21	30	3000.00	1
	3	2026-04-21	40	4200.00	2
	4	2026-04-22	25	2500.00	5
	5	2026-04-22	60	7200.00	3



DATABASE QUERIES

346

347 • SELECT Staff_Name, Contact_No

348 FROM Staff

349 WHERE Salary > 50000.00;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result

	Staff_Name	Contact_No
▶	Rohan Sharma	9876543201
	Priya Verma	9876543202
	Sneha Gupta	9876543204

Staff 6 x

QUERY: Find names and contact numbers of all staff members who earn a salary greater than 50,000

RELATIONAL

ALGEBRA : $\pi_{\text{staff-name}, \text{contact-no}} (\sigma_{\text{salary} > 50000} (\text{staff}))$

TRC:

$$\{t \mid \exists s \in \text{staff} (t[\text{staff-name}] = s[\text{staff-name}] \wedge t[\text{contact-no}] = s[\text{contact-no}] \wedge s[\text{salary}] > 50000)\}$$

DRC: $\{ \langle sn, c \rangle \mid \exists sid, t, s, bid (\langle sid, sn, c, t, s, bid \rangle \in \text{staff} \wedge s > 50000) \}$

```

355 • SELECT Customer_ID
356 FROM Customer
357 WHERE Customer_ID NOT IN (SELECT Customer_ID FROM Orders);
358

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

Customer_ID
19
11
9
18
14

Customer 7 x

QUERY: Find the IDs of customers who have never placed any orders

Relation Algebra: $\pi_{\text{customer-id}}(\text{Customer}) - \pi_{\text{customer-id}}(\text{Orders})$

TRC: $\{t \mid \exists c \in \text{Customer} (t[\text{customer-id}] = c[\text{customer-id}] \wedge \neg \exists o \in \text{Orders} (o[\text{customer-id}] = c[\text{customer-id}]))\}$

DRC: $\{ \langle \text{cid} \rangle \mid \exists \text{cn, e, p, a} (\langle \text{cid}, \text{cn}, \text{e}, \text{p}, \text{a} \rangle \in \text{Customer} \wedge \neg \exists \text{oid, bid, ot, st, am, ti} (\langle \text{oid}, \text{bid}, \text{ot}, \text{st}, \text{am}, \text{ti} \rangle \in \text{Orders})) \}$

```

424 • SELECT Staff_Name, Contact_No
425 FROM Staff
426 WHERE Type = 'Manager';

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Staff_Name	Contact_No
▶	Rohan Sharma	9876543201
	Neha Kapoor	9876543206
	Vikas Yadav	9876543211
	Kavya Reddy	9876543216

Staff 23 x

QUERY: Find the names and contact numbers of all staff members whose job type is 'Manager'.

Relational Algebra : $\pi_{\text{Staff-Name, Contact No}}(\sigma_{\text{Type}='Manager'}(\text{Staff}))$.

TRC : $\{t \mid \exists s \in \text{Staff} (t[\text{Staff-Name}] = s[\text{Staff-Name}] \wedge s[\text{Contact-No}] = t[\text{Contact-No}] \wedge s[\text{Type}] = 'Manager')\}$.

DRC : $\{ \langle sn, c \rangle \mid \exists sid, ty, s, bid (\langle sid, sn, c, ty, s, bid \rangle \in \text{Staff} \wedge ty = 'Manager') \}$

```

420 • SELECT Order_ID
421 FROM Order_Item
422 WHERE Quantity > 2;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Order_ID
5

Order_Item 22 x

QUERY: Find the order IDs where a single item's quantity in that order exceeded 2.

Relational Algebra: $\pi_{\text{Order-ID}}(\sigma_{\text{Quantity} > 2}(\text{Order-Item}))$

TRC: $\{t \mid \exists oi \in \text{Order-Item} (t[\text{Order-ID}] = oi[\text{Order-ID}] \wedge oi[\text{Quantity}] > 2)\}$.

DRC: $\{oid \mid \exists iid, q, (<oid, iid, q> \in \text{Order-Item} \wedge q > 2)\}$.

```

343
344 • SELECT o.Order_ID, o.Order_Type, c.Customer_Name
345 FROM Orders o
346 JOIN Customer c ON o.Customer_ID = c.Customer_ID;
347

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Order_ID	Order_Type	Customer_Name
▶	1	Dine-In	Aman Sharma
	2	Takeaway	Priya Verma
	3	Online	Rahul Singh
	4	Dine-In	Sneha Gupta
	5	Takeaway	Karan Mehta

Result 5 x

QUERY: Find the Order-ID, order-Type, customer-Name for all placed orders

Relational Algebra: $\pi_{\text{order-ID, order-Type, customer-Name}} (\text{orders} \bowtie \text{customers})$

TRC: $\{t \mid \exists o \in \text{orders} (\exists c \in \text{customers} (t[\text{order-ID}] = o[\text{order-ID}] \wedge t[\text{order-Type}] = o[\text{order-Type}] \wedge t[\text{customer-name}] = c[\text{customer-name}] \wedge o[\text{customer-ID}] = c[\text{customer-ID}]))\}$

DRC: $\{ \langle oid, ot, cn \rangle \mid \exists cid, bid, st, am, ti, e, p, a (\langle oid, cid, bid, ot, st, am, ti \rangle \in \text{orders} \wedge \langle cid, en, e, p, a \rangle \in \text{customers}) \}$

```

368 • SELECT Customer_Name AS Person_Name
369 FROM Customer
370 UNION
371 SELECT Staff_Name AS Person_Name
372 FROM Staff;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Person_Name
▶	Aman Sharma
	Priya Verma
	Rahul Singh
	Sneha Gupta
	Karan Mehta
	Neha Joshi
	Arjun Patel
	Simran Kaur
	Rohit Mehta

Result 9 x

QUERY : list all people's names in the database (combining customer and staff names)

Relational Algebra :

$$\rho_{\text{person_name}} (\pi_{\text{customer_name}} (\text{customer})) \cup$$

$$\rho_{\text{person_name}} (\pi_{\text{staff_name}} (\text{staff}))$$

TRC : $\{ t \mid \exists c \in \text{customer} (t[\text{Person_Name}] = c[\text{customer_Name}]) \vee \exists s \in \text{staff} (t[\text{Person_Name}] = s[\text{staff_Name}]) \}$

DRC : $\{ \langle pn \rangle \mid \exists cid, e, p, a (\langle cid, pn, e, p, a \rangle \in \text{customer}) \vee \exists sid, c, t, s, bid (\langle sid, pn, c, t, s, bid \rangle \in \text{staff}) \}$

```

409 • SELECT Item_Name
410 FROM Item
411 WHERE Item_ID NOT IN (SELECT Item_ID FROM Order_Item);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Item_Name

Item 15 x

QUERY: Find the names of items that are available in the menu but have never been ordered.

Relational Algebra : $\pi_{\text{Item_Name}}(\text{Item}) - \pi_{\text{Item_Name}}(\text{Order_Item} \bowtie \text{Item})$

TRC : $\{t \mid \exists i \in \text{Item} (t[\text{Item_Name}] = i[\text{Item_Name}] \wedge \sim \exists oi \in \text{Order_Item} (oi[\text{Item_ID}] = i[\text{Item_ID}]))\}$

DRC : $\{ \langle in \rangle \mid \exists iid, pr, av (\langle iid, in, pr, av \rangle \in \text{Item} \wedge \sim \exists oid, q (\langle oid, iid, q \rangle \in \text{Order_Item})) \}$

```

409 • SELECT i.Item_Name, ing.Name AS Ingredient_Name
410 FROM Item i
411 CROSS JOIN Ingredients ing
412 WHERE i.Price > (ing.Price * 2);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Item_Name	Ingredient_Name
Garlic Bread	Tomato
Chicken Wings	Tomato
Fried Rice	Tomato
Milkshake	Tomato
Mojito	Tomato
Brownie	Tomato
Ice Cream	Tomato
Spring Rolls	Tomato

Result 17 x

QUERY: Find combinations of items and ingredients where the price of the final item is more than 2 times the price of the individual ingredient

Relational Algebra:

$$\pi_{Item_Name, Name} (\sigma_{Item_Price > (Ingredients_Price \times 2)} (Item \times Ingredients))$$

$$TRC : \{t \mid \exists i \in Item (\exists ing \in Ingredients (t[Item_Name] \wedge i[Item_Name] \wedge t[Name] = ing[Name] \wedge i[Price] > (ing[Price] \times 2)))\}$$

$$DRC : \{ \langle in, ign \rangle \mid \exists iid, pr_i, av, igid, u, pr_{ig} (\langle iid, in, pr_i, av \rangle \in Item \wedge \langle igid, ign, u, pr_{ig} \rangle \in Ingredients \wedge pr_i > (pr_{ig} \times 2)) \}$$

```

375 • SELECT Branch_ID, Total_Cost
376 FROM Inventory_Order
377 WHERE Date > '2023-01-01';

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Branch_ID	Total_Cost
▶	1	5000.00
	1	3000.00
	2	4200.00
	5	2500.00
	3	7200.00
	2	3500.00
	4	5400.00
	1	2000.00

Inventory_Order 10 x

Output

QUERY: Find the Branch_ID and Total cost of all inventory orders placed by the branch after 1 Jan 2023.

Relational Algebra: $\pi_{\text{branch-id, total-cost}} (\sigma_{\text{date} > '2023-01-01'} (\text{Inventory_Order}))$

TRC: $\{ t \mid \exists io \in \text{Inventory_Order} (t[\text{branch-id}] = io[\text{branch-id}] \wedge t[\text{total-cost}] = io[\text{total-cost}] \wedge io[\text{date}] > '2023-01-01') \}$

DRC: $\{ \langle bid \rangle \mid \exists sid, sn, c, t, s (\langle sid, sn, c, t, s, bid \rangle \in \text{Staff}) \wedge \exists invid, supid, d, tc (\langle invid, supid, bid, d, tc \rangle \in \text{Inventory_Order}) \}$

```

379 • SELECT DISTINCT Branch_ID
380 FROM Staff
381 WHERE Branch_ID IN (SELECT Branch_ID FROM Inventory_Order);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Branch_ID
1
2
3
4
5

Staff 11 x

QUERY: Find the IDs of branch that currently employ staff and have placed inventory orders

Relational Algebra:

$$\pi_{\text{branch_id}}(\text{Staff}) \cap \pi_{\text{branch_id}}(\text{Inventory_Order})$$

$$\text{TRC: } \{ t \mid s \in \text{Staff} \wedge t[\text{branch_id}] = s[\text{branch_id}] \wedge \exists i \in \text{Inventory_Order} \wedge t[\text{branch_id}] = i[\text{branch_id}] \}$$

$$\text{DRC: } \{ \langle \text{bid} \rangle \mid \exists \text{sid}, \text{sn}, \text{c}, \text{t}, \text{s} (\langle \text{sid}, \text{sn}, \text{c}, \text{t}, \text{s} \rangle \in \text{Staff}) \wedge \exists \text{invid}, \text{supid}, \text{d}, \text{tc} (\langle \text{invid}, \text{supid}, \text{bid}, \text{d}, \text{tc} \rangle \in \text{Inventory_Order}) \}$$

```

359 |
360 • SELECT DISTINCT i.Item_Name
361 FROM Customer c
362 JOIN Orders o ON c.Customer_ID = o.Customer_ID
363 JOIN Order_Item oi ON o.Order_ID = oi.Order_ID
364 JOIN Item i ON oi.Item_ID = i.Item_ID
365 WHERE c.Customer_ID = 4;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Item_Name
Sandwich
Veg Wrap
Ice Cream

Result 8 x

Output

Query: Find the names of items ordered by the customer with customer ID '4'.

Relational Algebra: $\pi_{\text{item_name}}(\sigma_{\text{customer_ID}=4}(\text{customer} \bowtie \text{orders} \bowtie \text{order_Item} \bowtie \text{Item}))$

TRC: $\{t \mid \exists i \in \text{Item} (\exists oi \in \text{Order_Item} (\exists o \in \text{Orders} (t[\text{Item_name}] = i[\text{Item_Name}] \wedge i[\text{Item_ID}] = oi[\text{Item_ID}] \wedge oi[\text{Order_ID}] = o[\text{Order_ID}] \wedge o[\text{customer_ID}] = 4)))\}$

DRC: $\{ \langle in \rangle \mid \exists iid, pr, av, oid, q, bid, ot, st, om, ti (\langle iid, in, pr, av \rangle \in \text{Item} \wedge \langle oid, iid, q \rangle \in \text{Order_Item} \wedge \langle oid, 4, bid, ot, st, am, ti \rangle \in \text{orders}) \}$

```

391 • SELECT DISTINCT c.Customer_Name
392 FROM Customer c
393 JOIN Orders o ON c.Customer_ID = o.Customer_ID
394 WHERE c.Address = 'Delhi'
395 AND o.Amount > 400;

```

Result Grid | Filter Rows: | Export:

	Customer_Name
▶	Aman Sharma

Result 12 x

QUERY: find customers who live in Delhi and have placed orders with Amount > 400.

Relational Algebra: $\sigma_{\text{Address} = \text{'Delhi'} \wedge \text{Amount} > 400}$

$(\text{Customer} \bowtie \text{customer_customer_ID} = \text{Orders.customer_ID Orders})$

(only name): $\pi_{\text{customer_Name}} (\sigma_{\text{Address} = \text{'Delhi'} \wedge \text{Amt} > 400} (\text{Customer} \bowtie \text{Orders}))$

TRC: $\{c.\text{Customer_Name} \mid \text{Customer}(c) \wedge \exists o (\text{Orders}(o) \wedge c.\text{Customer_ID} = o.\text{Customer_ID} \wedge c.\text{Address} = \text{'Delhi'} \wedge o.\text{Amount} > 400)\}$

DRC: $\{c \text{ name} \mid \exists c'id, email, phone, addr, o'id, bi'd, o'type, status, amt, time (\text{Customer}(c'id, c'name, email, phone, addr) \wedge \text{Orders}(o'id, c'id, bi'd, o'type, status, amt, time))\}$

```

403 • SELECT p.Method
404 FROM Payment p
405 JOIN Orders o ON p.Order_ID = o.Order_ID
406 JOIN Branch b ON o.Branch_ID = b.Branch_ID
407 WHERE p.Amount > 100.00 AND b.Branch_Name = 'Central Hub';

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Method
UPI

Result 14 x

QUERY: Find the Payment methods used for orders placed at the branch 'Central Hub' where the payment amount was greater than 100.

Relational Algebra: $\pi_{\text{Method}} (\sigma_{\text{Amount} > 100 \wedge \text{Branch_Name} = \text{'Central Hub'}} (\text{Payment} \bowtie \text{Orders} \bowtie \text{Branch}))$

TRC: $\{t \mid \exists p \in \text{Payment} (\exists o \in \text{Orders} (\exists b \in \text{Branch} (t[\text{Method}] = p[\text{Method}] \wedge p[\text{Order_ID}] = o[\text{Order_ID}] \wedge o[\text{Branch_ID}] = b[\text{Branch_ID}] \wedge b[\text{Branch_Name}] = \text{'Central Hub'} \wedge p[\text{Amount}] > 100)))\}$

DRC: $\{ \langle m \rangle \mid \exists pid, oid, am-p, cid, bid, ot, st, am-o, ti, ba, ph, tm (\langle pid, oid, am-p, m \rangle \in \text{Payment} \wedge \langle oid, cid, bid, ot, st, am-o, ti \rangle \in \text{Orders} \wedge \langle bid, \text{'Central Hub'}, ba, ph, tm \rangle \in \text{Branch} \wedge am-p > 100) \}$

```

398 • SELECT S1.Staff_Name AS Staff_1, S2.Staff_Name AS Staff_2
399 FROM Staff S1
400 JOIN Staff S2 ON S1.Branch_ID = S2.Branch_ID
401 WHERE S1.Staff_ID < S2.Staff_ID;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Staff_1	Staff_2
Rohan Sharma	Raj Malhotra
Rohan Sharma	Pooja Nair
Rohan Sharma	Nisha Arora
Priya Verma	Neha Kapoor
Priya Verma	Ankit Jain
Priya Verma	Deepak Chauhan
Aman Singh	Simran Kaur
Aman Singh	Vikas Yadav

Result 13 x

QUERY: Find pairs of distinct staff members who work at the same branch.

Relational Algebra : $\pi_{S1.Staff_Name, S2.Staff_Name} (\sigma_{S1.Branch_ID = S2.Branch_ID \wedge S1[Staff_ID] < S2[Staff_ID]} (P_{S1}(Staff) \times P_{S2}(Staff)))$

Tuple Relational Calculus : $\{t \mid \exists s1 \in Staff (\exists s2 \in Staff (t[Staff_Name1] = s1[Staff_Name] \wedge t[Staff_Name2] = s2[Staff_Name] \wedge s1[Branch_ID] \wedge s1[Staff_ID] < s2[Staff_ID]))\}$

Domain Relational Calculus : $\{ \langle n1, n2 \rangle \mid \exists sid1, c1, t1, s1, bid, sid2, c2, t2, s2 (\langle sid1, n1, c1, t1, s1, bid \rangle \in Staff \wedge \langle sid2, n2, c2, t2, s2, bid \rangle \in Staff \wedge sid1 < sid2) \}$

Normalization

- Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.
- It is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data in different places), which otherwise unnecessarily increases the size of the database.
- Data redundancy leads to inconsistency problems during insert, delete, and update operations, making data management difficult.
- Normalization involves splitting a table into multiple tables, which must be linked each time a query requires data from the split tables.

Need of Normalization

Insertion Anomalies:

Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.

Deletion anomalies:

Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.

Updation anomalies:

Updation anomalies occur when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

Now, the goal is to generate a set of relation schemas that allows us to store information without unnecessary redundancy.

The approach is:

- * Decide if a given relation schema is in “good form.” There are a number of different forms (called normal forms), majorly we use 3NF and BCNF.
- * If a given relation schema is not in “good form,” then we decompose it into a number of smaller relation schemas, each of which is in an appropriate normal form. The decomposition must be a lossless decomposition.

Third Normal Form (3NF).

A relation is in the third normal form, if there is no transitive dependency for non-prime attributes as well as it is in the second normal form. A relation is in 3NF if at least one of the following conditions holds in every non-trivial function dependency $X \rightarrow Y$.

X is a super key

or

Y is a prime attribute (each element of Y is part of some candidate key).

Boyce-Codd Normal Form (BCNF).

For BCNF the relation should satisfy the below conditions

The relation should be in the 3rd Normal Form.

X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.



Analysing the relations for normal forms

Branch Table

The table tracks Branch_ID (Primary Key), Branch_Name, Branch_Address, Phone_no and Timing.

Functional Dependencies (FDs)

Branch_ID $\rightarrow$ Branch_ID, Branch_Name, Branch_Address, Timing

BUT, here phone no. is multi valued attribute. So, if we want to insert one more phone no. corresponding to a name, there is no functional dependency for that.

Hence, in this case we need to decompose the phone_no attribute to another table and here we come for the need of normalisation.

So, on decomposing we get two tables :

- The table tracks Branch_ID (Primary Key), Branch_Name, Branch_Address and Timing
- The table tracks Branch_ID (Foreign key) and Phone_no.

This table is a relation with composite key i.e foreign key + partial key and now both the decomposed tables are in BCNF.

Example of insertion anomaly for this Branch Table:

```
INSERT INTO branch (Branch_ID, Branch_Name, Branch_Address, Phone_No, Timing) VALUES
```

```
(1, 'North Branch', 'Delhi', '9876543211', '9 AM - 10 PM'),
```

```
(1, 'North Branch', 'Delhi', '9876500001', '9 AM - 10 PM'),
```

```
(2, 'South Branch', 'Delhi', '9876543212', '10 AM - 11 PM'),
```

```
(2, 'South Branch', 'Delhi', '9876500002', '10 AM - 11 PM'),
```

```
(3, 'East Branch', 'Delhi', '9876543213', '9 AM - 9 PM'),
```

```
(4, 'West Branch', 'Delhi', '9876543214', '8 AM - 10 PM'),
```

```
(5, 'Central Branch', 'Delhi', '9876543215', '10 AM - 10 PM'),
```

```
(6, 'City Branch', 'Delhi', '9876543216', '9 AM - 11 PM'),
```

Upon running the query:

✓	20	15:56:44	CREATE TABLE branch (Branch_ID INT PRIMARY KEY, Branch_Name VARCHAR(50), Branch_Add...	0 row(s) affected
✗	21	15:56:52	INSERT INTO branch (Branch_ID, Branch_Name, Branch_Address, Phone_No, Timing) VALUES (1, 'North Br...	Error Code: 1062. Duplicate entry '1' for key 'branch.PRIMARY'

And here the software gave error for inserting

Customer table

The table tracks Customer_ID (Primary Key), Customer_Name, Email (Unique), Phone (Unique), and Address.

Functional Dependencies (FDs)

- Customer_ID $\rightarrow$ Customer_ID ,Customer_Name, Email, Phone, Address
- Email $\rightarrow$ Customer_ID, Customer_Name, Email ,Phone, Address
- Phone $\rightarrow$ Customer_ID, Customer_Name, Phone ,Email, Address

3NF (Third Normal Form): There are no "transitive" dependencies where a non-key attribute depends on another non-key attribute.

BCNF (Boyce-Codd Normal Form): Every determinant (ID, Email, Phone) is a candidate key.
The table is in BCNF

Staff Table

The table tracks Staff_ID (Primary Key), staff_name, contact_no (Unique), Type, salary, and branch_ID (Foreign Key).

Functional Dependencies (FDs)

- Staff_ID $\rightarrow$ Staff_ID, staff_name, contact_no, Type, salary, branch_ID
- contact_no $\rightarrow$ Staff_ID, contact_no, staff_name, Type, salary, branch_ID
- 3NF (Third Normal Form): There are no transitive dependencies; every non-key attribute depends only on the candidate keys (Staff_ID and contact_no).
- BCNF (Boyce-Codd Normal Form): Every determinant (Staff_ID and contact_no) is a candidate key.

The table is in BCNF

Payment table

The table tracks Payment_ID (Primary Key), Order_ID (Foreign Key), Amount, and Method.

Functional Dependencies (FDs)

- Payment_ID $\rightarrow$ Payment_ID, Order_ID, Amount, Method
- 3NF (Third Normal Form): All non-key attributes (Order_ID, Amount, Method) depend only on the Primary Key. There are no transitive dependencies between non-key attributes.
- BCNF (Boyce-Codd Normal Form): The only determinant is Payment_ID, which is the Primary Key and therefore a candidate key.

Final Result:

The table is in BCNF.

Order table

The table tracks Order_ID (Primary Key), Customer_ID (Foreign Key), Branch_ID (Foreign Key), Order_Type, Status, Amount, and Time.

Functional Dependencies (FDs)

- $\text{Order_ID} \rightarrow \text{Order_ID}, \text{Customer_ID}, \text{Branch_ID}, \text{Order_Type}, \text{Status}, \text{Amount}, \text{Time}$
- 3NF (Third Normal Form): There are no transitive dependencies; all non-key attributes (Customer_ID, Branch_ID, Order_Type, Status, Amount, Time) depend directly on the Primary Key.
- BCNF (Boyce-Codd Normal Form): The only determinant is Order_ID, which is the Primary Key and a candidate key.

The table is in BCNF.

Item table

- The table tracks Item_ID (Primary Key), Item_Name, Price, and Availability.

Functional Dependencies (FDs)

- $\text{Item_ID} \rightarrow \text{Item_ID}, \text{Item_Name}, \text{Price}, \text{Availability}$
- 3NF (Third Normal Form): Every non-key attribute (Item_Name, Price, Availability) depends only on the Primary Key. There are no dependencies between non-key attributes.
- BCNF (Boyce-Codd Normal Form): The determinant (Item_ID) is a candidate key.
- The table is in BCNF.

Order items table

The table tracks Order_ID (Composite Primary Key), Item_ID (Composite Primary Key), and Quantity. Both ID columns also serve as Foreign Keys.

Functional Dependencies (FDs)

- $(\text{Order_ID}, \text{Item_ID}) \rightarrow \text{Order_ID}, \text{Item_ID}, \text{Quantity}$
- 3NF (Third Normal Form): There are no transitive dependencies; the only non-key attribute (Quantity) depends directly on the entire composite Primary Key.
- BCNF (Boyce-Codd Normal Form): The determinant (Order_ID, Item_ID) is the composite candidate key.

The table is in BCNF.

Ingredients table

The table tracks ID (Primary Key), Name, Unit, and Price.

Functional Dependencies (FDs)

- $ID \rightarrow Name, ID, Unit, Price$
- 3NF (Third Normal Form): Every non-key attribute (Name, Unit, Price) depends directly on the Primary Key. There are no transitive dependencies between non-key attributes.
- BCNF (Boyce-Codd Normal Form): The determinant (ID) is a candidate key.

The table is in BCNF.

Inventory orders table

The table tracks inventory_ID (Primary Key), date, quantity, total_cost, and branch_ID (Foreign Key).

Functional Dependencies (FDs)

- $\text{inventory_ID} \rightarrow \text{inventory_ID}, \text{date}, \text{quantity}, \text{total_cost}, \text{branch_ID}$
- 3NF (Third Normal Form): Every non-key attribute (date, quantity, total_cost, branch_ID) depends directly on the Primary Key. There are no transitive dependencies between non-key attributes.
- BCNF (Boyce-Codd Normal Form): The only determinant is inventory_ID, which is a candidate key.

The table is in BCNF.

thank
you